Object Oriented Programming for Data Processing
20 January 2025

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 14:15 as a single `zip` file.

C++ evaluation will be based on: correct syntax, proper return types and arguments of functions, data members and class interfaces, setters/getters, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++ (RLC circuits)

- Create an abstract class `CircuitElement` and 3 concrete classes that inherit from it:

  1. `Resistor`, with an attribute to represent the resistance R;
  2. `Inductor`, with an attribute to represent the inductance L;
  3. `Capacitor`, with an attribute to represent the capacitance C.

  Make sure the user can know at any time the number of resistor, inductor, and capacitor instances. Further, overload the `+` and `||` operators to combine pairs of resistors, inductors, or capacitors in series and in parallel, respectively. [Reminder: resistors and inductors in series add up, as do capacitors in parallel; in all other cases, the equivalent element parameter is given by $(x_1^{-1} + 1/x_2^{-1})^{-1}$].

- Create an abstract class called `RLCFilter`, which uses instances of the 3 concrete classes above to represent a resistor, an inductor, and an capacitor. These filters are characterized by a distinctive angular frequency $\omega_c = 1/\sqrt{LC}$, a bandwidth (or stopband) $\Delta\omega$, and a quality factor $Q = \omega_c/\Delta\omega$. Also create the following 4 concrete classes that inherit from `RLCFilter` to represent different RLC circuit topologies:

  1. `HighPass`, where $\Delta\omega = \omega_c$ (and ω_c is known as corner frequency);
  2. `LowPass`, that stores $\Delta\omega = \omega_c$ and the damping factor $\zeta = \sqrt{L/C}/(2R)$;
  3. `BandPassSeries`, where $\Delta\omega = R/L$ (and ω_c is known as center frequency);
  4. `BandPassShunt`, where $\Delta\omega = 1/(CR)$;

  All classes must have getters for all their data members, adequate setters for R, L, and C, and they must overload the `operator<<` to be used with `std::cout`.

Part 2 – Python

Build a class to handle the following system of two coupled third-order differential equations:

$$\begin{cases} \dddot{x_2} = -\ddot{x_1}^3 + \dot{x_2} + x_1 + \sin(t) \\ \dddot{x_1} = -2\dot{x_2}^2 + x_2 \end{cases},$$

where $t \geq 0$ is the independent variable, and x_1 and x_2 depend upon t. [Reminder: you need to introduce auxiliary, intermediate variables in order to deal with a higher number of first-order differential equations.]

Use a Python notebook or Python scripts to complete the following tasks.

1. Provide the ability to initialize instances of the class by passing the initial conditions for x_1 and x_2, and their derivatives up to second order.

2. Provide a method to simulate the evolution of $x_1(t)$ and $x_2(t)$ for a time T with a sampling time Δt. The resulting $\{t_i, x_1(t_i), x_2(t_i)\}$ values must be saved to a `pickle` file.

3. Provide the ability to display the results of a simulation by generating 2 plots: one with $x_1(t)$ and its first and second derivatives as a function of t, and a similar one for $x_2(t)$ and its derivatives.

4. Finally, write a second, more general version of your class, where the $-2\dot{x_2}^2$ term in the second equation is of the form $\alpha\dot{x_2}^2$ with the parameter α specified by the user when initializing an instance of the class, and similarly $\sin(t)$ in the first equation is replaced by a t-dependent function also provided by the user.